

第四节

初始化 Agent OS 代码框架，搭建开源官网

前三节建立了认知地图。这节课开始动手。
用 Spec-Kit 把需求文档和技术方案转成 AI 可以直接执行的任务，
用 Subagents 并行初始化 OryxOS 项目骨架，顺手把官网搭起来。



> @Robert/Oryx Labs

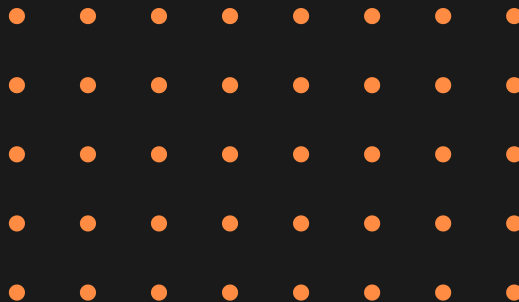
```
bash - 80x24

$ /specify
  ↳ AI 追问边界条件，输出可执行规格

$ /plan
  ↳ 生成技术方案，等你 review 确认

$ /tasks
  ↳ 拆解为独立可验收任务列表

$ /implement
  ↳ 逐任务执行，每步可见可控
```



今天跑通 OryxOS 的"第一公里"

三周训练营，这节课是 Day 1。结束时你手里有真实可运行的东西。

01 Spec-Kit 起手

constitution + spec + plan + tasks

把需求文档和技术方案喂给 Spec-Kit，跑出完整的 AI 编程工作包。这是后续所有实施的地基，做一次管三周。

02 constitution.md 实战

6 条原则，不可违背

把工程原则变成 AI 必须遵守的硬约束。看清楚有 constitution 和没有 constitution，AI 写出来的代码差在哪里。

03 Subagents 并行初始化

4 个 subagent 同时跑 4 个模块

用 Claude Code 的 Subagents 机制，4 个并行 subagent 同时初始化 Maven 多模块骨架。30 分钟内 11 个模块编译通过。

04 搭建开源官网

VitePress + GitHub Pages

用 AI 编程 30 分钟做出 OryxOS 官网并部署上线。开源项目的门面，让别人看到这个项目是认真的。

Spec-Kit 是什么：为什么用它起手 OryxOS

GitHub 2026 年开源的 spec-driven development 工具链，90,000+ stars
把需求文档转成 AI agent 能直接消费的任务格式，强制 spec 先行，防止 AI
vibe coding 跑偏

为什么 OryxOS 用 Spec-Kit

OryxOS 已经有了完整的需求文档和技术方案。Spec-Kit 的工作是把这两份文档转成 AI 能直接执行的格式，不重写。投入不浪费，文档直接变成 AI 的工作包。

Spec-Kit 适合什么场景

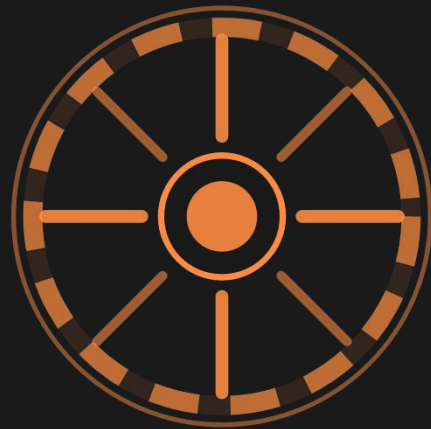
Medium 到 large 规模的 greenfield 项目（OryxOS 是 11 个 Maven 模块的新项目，完全匹配）。需求清晰、AI agent 协作、强制方法论这三条 OryxOS 全中。

为什么不从增量阶段就用 Spec-Kit

Spec-Kit 对小增量过重——一次修 1-2 个文件，跑完整的 constitution + specify + plan + tasks 流程，开销大于任务本身。主体开发用 Spec-Kit，增量开发切换到手动提示词。

跟需求文档和技术方案的关系

需求文档 → specify 的输入。技术方案 → plan 的输入。需求文档第 3 章 + 技术方案第 1.1 节 → constitution 的来源。文档是输入，Spec-Kit 是转换器。



Spec-Kit 跟 OryxOS 的契合度：纯 greenfield + 需求清晰 + AI agent 协作 + 方法论场景，每条都对得上

Spec-Kit 四步流程：从文档到可执行任务

/speckit.constitution

→ constitution.md

6 条不可违背的原则

- JDK 21 + Spring Boot 3.x
- 自实现 ReAct loop
- Plugin Tool 三档接入
- SQLite + MEMORY.md
- 单二进制部署
- 跑通优先于完美

/speckit.specify

→ spec.md

5 个 User Story

- US-1: 对接 LLM (P1)
 - US-2: ReAct 循环 (P1)
 - US-3: Memory (P2)
 - US-4: Tool 体系 (P2)
 - US-5: Web Service (P3)
- 每个 US 有验收 demo

/speckit.plan

→ plan.md

技术方案转实施计划

- 11 个 Maven 模块划分
- 关键技术决策展开
- 模块间协作关系
- 数据流设计
- 3周里程碑时间线

/speckit.tasks

→ tasks.md

每个 US 拆分 task

- 1-2个文件/task 颗粒度
- 可并行 task 标出来
- 依赖关系排序
- 每 task 完成有可运行代码

跑完四步，OryxOS 1.0 的完整工作包就绪。后续每个 US 按 tasks.md 逐 task 实施，AI agent 每次只看一个 task，不会跑偏。

constitution.md: 工程宪章, 不是写给人看的

constitution.md 里的每一条, AI agent 在 implement 阶段都会主动引用。它不是设计文档, 是 AI 的行为约束。

没有 constitution

- AI 按自己理解选 JDK 17, 不是 JDK 21
- 直接用 Spring AI 的 Agent 抽象, 不自己实现 ReAct
- 用 H2 数据库, 不用 SQLite
- Plugin Tool 只做了 @Tool 注解方式, SKILL.md 和 MCP 没做
- 把五大能力揉进一个模块, 没有按核心能力分模块
- 没有单二进制部署配置

结果: review 发现问题, 返工, 浪费 2-3 小时

✓ 有 constitution

- AI 实施前主动读 constitution, JDK 21 是硬要求
- ReAct loop 必须自己实现, AI 不会走捷径
- SQLite 是 constitution 明确指定的
- Plugin Tool 三档都在 constitution 里, 缺一条 AI 会问
- 11 个 Maven 模块在 plan.md 里, constitution 要求遵守
- GraalVM Native Image 是扩展目标, constitution 标注清楚

结果: 第一次就对, 不需要返工

OryxOS constitution.md: 6 条不可协商原则

写一次，整个主体开发期间不改。如果发现某条不对，停下来讨论，不允许 AI agent 自己修改。

原则一 技术栈锁定

JDK 21 + Spring Boot 3.x 单体应用，Maven 多模块，单二进制部署。
GraalVM Native Image 在扩展阶段引入，核心阶段先用标准 Spring Boot。

原则二 五大核心能力优先

LLM / ReAct / Memory / Tool / Web Service 五大能力是主线，支撑模块次之。不允许为了支撑模块的完整性牺牲核心能力的跑通。

原则三 ReAct loop 自己实现

不直接用 Spring AI 的 Agent 抽象。ReAct loop 自己写，不超过 100 行 Java。让学员能完整理解 Agent 工作机制，也保留未来定制循环行为的空间。

原则四 Plugin Tool 三档接入

必须同时支持零代码（SKILL.md + MCP）、轻代码（MCP server）、重代码（@Tool 注解）三档。主推零代码方式，其余两档不能缺。

原则五 存储方案固定

核心阶段：SQLite + Spring Data JPA 做关系型数据，MEMORY.md 文件做长期记忆，关键词检索。向量检索放扩展阶段，不提前引入。

原则六 跑通优先于完美

每个 user story 的目标是"可演示 demo"，不是"生产完美代码"。代码能跑通、demo 能演示，就算完成。性能优化、边界处理放扩展阶段。

Subagents 并行初始化：从一个 AI 到一个 AI 团队

OryxOS 有 11 个 Maven 模块。如果串行初始化每个模块，一个一个来，要花多少时间？
Subagents 让这件事并行。

串行方式

1. 主对话初始化 oryxos-core
2. 等待完成，主对话上下文增长
3. 初始化 oryxos-provider
4. 等待完成，继续累积上下文
5. 初始化 oryxos-memory
6. ...11 个模块一个一个来

问题：主对话上下文越来越长，LLM 注意力稀释，后面的模块质量下降

Subagents 并行

主 Agent 把任务分发给 4 个 Subagent

- Subagent A → core + provider 模块
- Subagent B → memory + tool-builtin
- Subagent C → tool-skill + tool-mcp
- Subagent D → web + storage + cli + boot

4 个 Subagent 独立运行，互不污染上下文

优势：每个 Subagent 上下文干净，质量稳定。速度是串行的 4 倍。

4 个 Subagent 分工：11 个模块并行初始化

Maven 多模块骨架按核心能力划分，4 组并行，每组上下文独立。

Subagent A

oryxos-core

核心抽象：ReActLoop /
PromptBuilder /
ToolExecutor / OryxTool /
Session / Profile

oryxos-provider

核心能力一：ProviderService
+ Function Calling 适配，基
于 Spring AI Alibaba

Subagent B

oryxos-memory

核心能力三：MemoryService
+ MemoryTools
(save_memory +
recall_memory)

oryxos-tool-builtin

核心能力四内置：FileTools /
ShellTools / HttpTools /
SandboxChecker

Subagent C

oryxos-tool-skill

核心能力四 Plugin 方式一：
SkillLoader，加载 SKILL.md
注入 system prompt

oryxos-tool-mcp

核心能力四 Plugin 方式二：
McpClientService +
McpToolAdapter

Subagent D

oryxos-web

核心能力五：WebServer + 6
个 ApiController +
GlobalExceptionHandler

oryxos-storage

SQLite 存储层：
SessionRepository (Spring
Data JPA)

oryxos-channel-cli

CLI Channel: oryxos chat 命
令实现

oryxos-cli + oryxos-boot

Picocli 12 个子命令 + Spring
Boot 启动模块 (fat JAR 打包)

每个 Subagent 完成时：Maven 模块目录 + pom.xml + 核心类骨架 + 基础测试。
mvn compile 通过。

Subagents 跑完：你手里有什么

不是 demo 代码，不是玩具项目。是 OryxOS 1.0 真实可编译的 Maven 多模块骨架。

oryxos/ 项目结构

pom.xml ← 父 pom，管理依赖
oryxos-core/ ← 核心抽象
 └─ ReActLoop.java ← < 100行
oryxos-provider/ ← LLM 接入
oryxos-memory/ ← Memory
oryxos-tool-builtin/ ← 内置工具
oryxos-tool-skill/ ← SKILL.md
oryxos-tool-mcp/ ← MCP Client
oryxos-web/ ← REST API
oryxos-storage/ ← SQLite
oryxos-channel-cli/ ← CLI
oryxos-cli/ ← Picocli
oryxos-boot/ ← fat JAR



mvn compile 通过

所有 11 个模块编译成功，没有依赖冲突，pom.xml 配置正确



核心接口定义好

OryxTool / Session / Profile / ReActLoop 等核心抽象的接口定义完成



Spring Boot 能启动

oryxos-boot 能打成 fat JAR，java -jar 启动不报错



constitution 全部满足

跑 /speckit.analyze 检查代码和 constitution 一致性，0 个漂移

这是后续 3 周实施的起点。US-1 到 US-5 按 tasks.md 逐 task 实施，每次 AI 只看一个 task。

开源官网：为什么项目必须有一个对外的门面

第一印象

有人看到 OryxOS 的第一个动作是：打开 GitHub → 点 Website 链接。如果没有官网，或者只有一个 README，这个项目看起来不认真。开源项目的门面，决定了第一批星星来不来。

文档入口

官网不只是介绍页，是文档体系的入口。What is OryxOS / Quick Start / API Reference / 部署指南，这些内容放官网上才能被 Google 索引到，才能被搜到。只在 GitHub README 里放文档，是最低效的分发方式。

社区信任

官网是项目认真程度的信号。Hermes Agent 有 hermes-agent.nousresearch.com，OpenClaw 有 openclaw.ai，Spring AI 有 docs.spring.io/spring-ai。OryxOS 定位企业场景，官网是建立信任的第一步。

贡献者入口

社区贡献者在哪里找贡献指南？在官网。CONTRIBUTING.md 链接到哪里？官网。训练营学员做的文档 PR，最终展示在哪里？官网。官网是社区的公告栏。

OryxOS 定位企业场景，官网是第一个可以公开分享的证明——“这个项目是认真的”



技术路径：VitePress + GitHub Pages

跟 mq9.robustmq.com 完全一样的路径。VitePress 生成静态站，GitHub Actions 自动部署。

01

写 Markdown

index.md (首页)
docs/ 目录 (文档)
public/ 目录 (SVG 图)

02

配置 VitePress

.vitepress/config.mts
s
导航 / 侧边栏 / 主题
SEO meta 标签

03

本地预览

npm run dev
实时看效果
确认内容和样式

04

GitHub Actions 部署

push 触发 workflow
vitepress build
自动推送到 gh-
pages

05

上线访问

绑定自定义域名
写 public/CNAME
约 2 分钟后可访问

项目结构

website/index.md .vitepress/config.mts public/favicon.svg public/diagram-*.svg docs/what.md
docs/quick-start.md .github/workflows/deploy.yml

OryxOS 官网结构：今天做出来的样子

导航结构

首页 (Home) Hero 区: 是什么 / 为什么 Java / 快速开始

Docs (文档) What is OryxOS · Quick Start · Architecture · Five Capabilities

Blogs (博客) 指向 Oryx 社区博客

GitHub (源码) github.com/oryx-labs/oryxos

- ▶ 侧边栏 (Docs 页)
- ▶ Getting Started
- ▶ Core Concepts
- ▶ Integration
- ▶ Deployment

首页 Hero 区

大标题: "Enterprise Agent OS for Java Developers"

副标题: "Run and manage AI Agents on your own infrastructure. Built on Spring Boot. No vendor lock-in."

CTA 按钮: "Quick Start" (橙色) / "GitHub" (次要)

三个亮点卡片:

- ① Java Native (Spring Boot 原生)
- ② Private (数据不出企业)
- ③ Extensible (Plugin Tool 三档)

技术栈 logo 行: Java · Spring Boot · Spring AI
Alibaba · MCP · SQLite

底部 tagline: "Brought to you by Oryx Labs"

用 AI 编程做官网：30 分钟内上线

不是手写每一行 markdown，是用 AI 生成内容，人来审核和调整结构。

Step 1 让 AI 读懂项目

把业界调研 + 需求文档 + 技术方案喂给 Claude。让它理解 OryxOS 是什么、跟竞品有什么不同、目标用户是谁。这是写官网内容的前提。

Step 2 生成 index.md

让 Claude 写首页 markdown，包括 hero 标题、副标题、三个亮点卡片、快速开始引导。参考 mq9 的 index.md 格式，但内容完全是 OryxOS 的。

Step 3 生成 config.mts

让 Claude 配置 VitePress 的 title / description / nav / sidebar / head meta。导航结构按上一页设计好的填入。SEO 标签一并生成。

Step 4 生成架构 SVG 图

让 Claude 画 OryxOS 五层架构图的 SVG 版本（简化版，适合官网展示），放进 public/ 目录。官网里 `` 直接引用。

Step 5 GitHub Actions 部署

复用 mq9 的 deploy.yml workflow，修改几行参数。push 到 main 分支，Actions 自动跑 vitepress build，推到 gh-pages，2 分钟后可访问。

这个流程不只适合 OryxOS。任何开源项目，这套模板 30 分钟复制一份。

```
bash - 80x24

$ /specify
  ↳ AI 追问边界条件，输出可执行规格

$ /plan
  ↳ 生成技术方案，等你 review 确认

$ /tasks
  ↳ 拆解为独立可验收任务列表

$ /implement
  ↳ 逐任务执行，每步可见可控
```

今天结束，你手里有什么

01 constitution.md

工程宪章，管三周

6 条不可违背的原则，后续所有 AI 编程的约束基础。只要按 constitution 做，代码不会跑偏。

02 spec.md + plan.md + tasks.md

完整工作包，可以直接实施

5 个 User Story 拆解完毕，每个 task 1-2 个文件颗粒度。后续三周就是按 tasks.md 逐 task 实施。

03 OryxOS Maven 骨架

11 个模块，mvn compile 通过

不是空壳，是有核心接口定义、有基础测试、可以 Spring Boot 启动的真实骨架。这是 US-1 到 US-5 实施的起点。

04 OryxOS 官网上线

GitHub Pages 可访问

有 URL，有内容，有架构图，有快速开始文档。任何时候都可以分享这个链接，让别人第一眼就看到这个项目是认真的。



四节课走完：认知地基打完，训练营正式开始

第一节

AI 编程的本质、工具、方法论体系

程序员目的没变，路径变了三层。七层工具架构。SDD + Harness 是当下最完整的方法论组合



第二节

AI 时代核心竞争力和 Oryx 社区

三层竞争力：用熟 AI + 工程纪律 + 复利资产。开源是最好的载体，AI 时代红利在放大。Oryx 社区四个项目



第三节

Agent OS 是什么，如何设计实现

五组件拆解 Agent。OpenClaw vs Hermes 对比。OryxOS = Java + 企业场景 + 私有部署 + day one 企业级治理



第四节

初始化代码框架，搭建开源官网

Spec-Kit 完整工作包就绪。Subagents 并行 11 个模块编译通过。OryxOS 官网上线。手里有真实可运行的东西



认知地图有了 · 竞争力方向清楚了 · 项目技术本质看透了 · 代码骨架跑起来了 · 官网门面立起来了
这四节课，是后面十周最值得的前置投入。训练营真正的内容，从下周开始。